

Solution

Q1 Solution

Solution:

1. The described behavior indicates that the database system follows the **steal policy. Justification**

In a steal policy, modified pages can be removed from the buffer and written to disk before the transaction commits. In this scenario, T1's updated page was written to disk before the transaction committed, which is a characteristic of the steal approach. Conversely, in a no-steal policy, dirty pages are not written to disk until the transaction commits, ensuring that uncommitted changes never appear on disk.

2. The system crashes before T1's log records are complete. This means the disk contains partial or inconsistent changes made by T1.

Undo Phase:

Since T1 did not commit, all changes made by T1 need to be undone to restore the database to its previous consistent state. Any updates written to disk by T1 are overwritten with their pre-update values.

Redo Phase (if necessary):

If other committed transactions' changes were not fully written to disk before the crash, the system re-applies their updates using the log.

0. Advantages:

The steal policy allows the buffer manager to remove dirty pages freely, making better use of available memory. This avoids situations where the buffer is filled with uncommitted changes, improving system scalability and performance.

Long-running transactions can hold a lot of data in the buffer without waiting for the transaction to commit.

Q2 Solution

Deadlock Scenario:

1. **T1** acquires a **write lock** on Account A to withdraw \$300.
2. **T2** acquires a **write lock** on Account B to withdraw \$200.

3. **T1** tries to acquire a **write lock** on Account B but is blocked because T2 already holds it.
4. **T2** tries to acquire a **write lock** on Account A but is blocked because T1 already holds it.

Now, both transactions are waiting for each other indefinitely, leading to a **deadlock**.

Solution – Deadlock Prevention Using Wait-Die Scheme:

1. Assign timestamps to the transactions.
 - Suppose **T1** has TS = 5 and **T2** has TS = 8.
2. When a transaction requests a lock held by another:
 - If the requesting transaction is **older** (lower TS), it **waits**.
 - If the requesting transaction is **younger** (higher TS), it is **aborted** and restarted.

Applying the Wait-Die Scheme:

- When T1 requests Account B (held by T2):
 - T1 waits because it is older (TS = 5).
- When T2 requests Account A (held by T1):
 - T2 is aborted because it is younger (TS = 8).

T2 restarts, avoiding the deadlock.

Q3 Solution

User A adds an item to their cart but the system crashes before the transaction is completed.

● **Atomicity:** Since the transaction did not complete, the system must ensure that no partial changes (such as the item being added to the cart) remain. The recovery mechanism will roll back the transaction to maintain atomicity.

2. User B places an order, which updates the inventory, but another user, User C, views the inventory before the update is committed.

● **Isolation:** The system must isolate the transaction of User B until it is committed. User C

should see the inventory state as it was before User B's update. This prevents dirty reads and ensures consistent inventory visibility for other users.

3. User D makes a payment, but due to a system failure, their payment status is not saved even though the payment is processed by the bank.

- **Durability:** Once the payment transaction is committed, the system must ensure that the changes are permanently recorded in the database, even in the event of a system crash.

Durability ensures that the payment status reflects the completed transaction.

Consistency Preservation:

In all cases, the database must ensure that it transitions from one consistent state to another.

For example:

- Inventory updates should respect stock constraints.

- Payment transactions should align with the actual processed payments.

The system's integrity constraints and business logic help enforce consistency.

Q4 Solution

Timestamp Ordering Protocol:

- Transactions are assigned unique timestamps (TS) when they start.
- Operations are executed based on these timestamps to ensure serializability.
- **Read Rule:** A transaction can read a data item X if $TS(T) \geq WTS(X)$.
- **Write Rule:** A transaction can write to X if $TS(T) \geq RTS(X)$ and $TS(T) \geq WTS(X)$.

Challenges in Basic Timestamp Ordering:

- **Cascading Rollbacks:** If a transaction T1 writes and is later aborted, all transactions that depend on T1's writes (e.g., T2) must also be aborted. This leads to performance issues.

How Strict Timestamp Ordering Addresses These Challenges:

- **Strict Timestamp Ordering** delays writes until a transaction commits. This ensures that only committed values are read, preventing cascading rollbacks.

Example:

- **Basic Timestamp Ordering:**
T1 (TS=5) writes X, T2 (TS=10) reads X. If T1 is aborted, T2 must be rolled back.
- **Strict Timestamp Ordering:**
T1 (TS=5) writes X, but the write is delayed until T1 commits. If T1 aborts, T2 is unaffected because it reads only committed values.

This avoids cascading rollbacks.

Q5 Solution

a) R1(X); R2(Y); W1(Y); C1; A2

- Transaction 2 reads data after Transaction 1 writes Y.
 - **Recoverable:** Yes, T2 aborts, and T1 commits before T2.
 - **Cascading rollback:** No, T2 doesn't depend on uncommitted data.
 - **Cascadeless:** Yes, T2 reads only committed data.
 - **Strict:** Yes, no data is read/written from uncommitted data.
 - **Conflict serializable:** Yes (T1 → T2).
-

b) R1(X); W1(X); R2(X); W2(X); C2; C1

- Transaction 2 writes X after reading and writing X by Transaction 1.
 - **Recoverable:** Yes, T1 commits after T2.
 - **Cascading rollback:** No, no rollback propagation occurs.
 - **Cascadeless:** No, T2 reads uncommitted X.
 - **Strict:** No, T2 reads uncommitted X.
 - **Conflict serializable:** Yes (T1 → T2).
-

c) R1(X); W1(X); R2(X); W2(X); A1; A2

- Transaction 2 writes X after Transaction 1 writes X.
 - **Recoverable:** Yes, no committed transaction reads uncommitted data.
 - **Cascading rollback:** Yes, aborting T1 causes T2 to rollback.
 - **Cascadeless:** No, T2 reads uncommitted X.
 - **Strict:** No, T2 reads uncommitted X.
 - **Conflict serializable:** Yes (T1 → T2).
-

d) W1(X); R2(X); A1; W2(X); C2

- Transaction 2 reads and writes X after T1 writes it.
 - **Recoverable:** No, T2 commits after T1 aborts.
 - **Cascading rollback:** No, T2 is not aborting.
 - **Cascadeless:** No, T2 reads uncommitted X.
 - **Strict:** No, T2 reads uncommitted X.
 - **Conflict serializable:** Yes (No cycle)
-

e) R1(X); W1(X); R2(X); C1; W2(X); C2

- T2 reads and writes X after T1 writes and commits X.
 - **Recoverable:** Yes, T1 commits before T2 commits.
 - **Cascading rollback:** No, no rollback propagation occurs.
 - **Cascadeless:** No, T2 does not read committed X.
 - **Strict:** No, T2 writes only committed X but the read is uncommitted.
 - **Conflict serializable:** Yes (T1 → T2).
-

f) W1(X); R2(X); W2(X); A2; C1

- T2 reads X written by T1 but writes to it before committing.

- **Recoverable:** Yes, T1 commits after T2 aborts.
 - **Cascading rollback:** No, aborting T2 does not cause any dependency issues.
 - **Cascadeless:** No, T2 reads uncommitted X.
 - **Strict:** No, T2 reads uncommitted X.
 - **Conflict serializable:** Yes ($T1 \rightarrow T2$).
-

g) W1(X); W2(Y); R2(X); R1(Y); W1(Y); C1; C2

- T2 reads X written by T1 and T1 reads Y written by T2.
 - **Recoverable:** Yes, no uncommitted data is committed prematurely.
 - **Cascading rollback:** No, no rollback propagation occurs.
 - **Cascadeless:** No, uncommitted data is read.
 - **Strict:** No, uncommitted data is read or written.
 - **Conflict serializable:** No ($T1 \leftrightarrow T2$ cyclic dependency).
-

h) R1(X); W1(X); R2(X); A1; W2(X); C2

- T2 writes X after T1 writes it and then aborts.
 - **Recoverable:** No, T1 aborts before T2 commits.
 - **Cascading rollback:** No, aborting T1 affects T2.
 - **Cascadeless:** No, T2 reads uncommitted X.
 - **Strict:** No, T2 reads uncommitted X.
 - **Conflict serializable:** Yes ($T1 \rightarrow T2$).
-

i) W1(X); W2(Y); W1(Y); C1; R2(X); C2

- T2 reads X written by T1 and T1 writes Y after T2 writes it.
- **Recoverable:** Yes, T1 commits before T2 reads X.
- **Cascading rollback:** No, no rollback propagation occurs.

- **Cascadeless:** Yes, no uncommitted data is read.
 - **Strict:** No, uncommitted data is written.
 - **Conflict serializable:** No($T1 \leftrightarrow T2$, cyclic dependency).
-

j) W1(X); W2(Y); R1(Y); R2(X); A1; A2

- T1 and T2 read and write each other's data.
- **Recoverable:** Yes, no uncommitted data is prematurely committed.
- **Cascading rollback:** Yes, aborting T1 aborts T2.
- **Cascadeless:** No, uncommitted data is read.
- **Strict:** No, uncommitted data is read.
- **Conflict serializable:** Yes ($T1 \leftrightarrow T2$ cyclic dependency).

Q6 Solution

a. Display all courses for which Professor Miley has been qualified.

$\pi_{\text{CourseName}}((\sigma_{\text{ProfName}='Miley'}(\text{Faculty}) \bowtie \text{Qualified}) \bowtie \text{Course})$

SELECT DISTINCT c.CourseId, c.CourseName FROM Faculty f JOIN Qualified q ON f.FacultyId = q.FacultyId JOIN Course c ON q.CourseId = c.CourseId WHERE f.ProfName = 'Miley';

b. Which instructors are qualified to teach CT220

$\pi_{\text{ProfName}}((\sigma_{\text{CourseId}='CT220'}(\text{Qualified})) \bowtie \text{Faculty})$

SELECT DISTINCT f.ProfName FROM Faculty f JOIN Qualified q ON f.FacultyId = q.FacultyId WHERE q.CourseId = 'CT220';

c. Is any instructor qualified to teach CT220 and not qualified to teach CS208?

$\pi_{\text{ProfName}}((\pi_{\text{FacultyId}}(\sigma_{\text{CourseId}='CT220'}(\text{Qualified})) - \pi_{\text{FacultyId}}(\sigma_{\text{CourseId}='CS208'}(\text{Qualified}))) \bowtie \text{Faculty})$

SELECT DISTINCT f.ProfName FROM Faculty f JOIN Qualified q1 ON f.FacultyId = q1.FacultyId LEFT JOIN Qualified q2 ON q1.FacultyId = q2.FacultyId AND q2.CourseId = 'CS208' WHERE q1.CourseId = 'CT220' AND q2.FacultyId IS NULL;

d. How many students are enrolled in section 1345 during semester I-2012

$\delta\text{StudentNo}(\pi\text{StudentNo}(\sigma\text{SectionID}=1345 \text{ and Semester}=\text{'I-2012'}(\text{Registration})))$

SELECT COUNT(DISTINCT r.StudentNo) AS TotalStudents FROM Registration r WHERE
r.SectionID = 1345 AND r.Semester = 'I-2012';

e. How many students are enrolled in CT220 during semester I-2012?

$\delta\text{StudentNo}(\pi\text{StudentNo}(\sigma\text{Semester}=\text{'I-2012'}(\pi\text{SectionNo}(\sigma\text{CourseId}=\text{'CT220'}(\text{Section}))\bowtie\text{Registration})))$

SELECT COUNT(DISTINCT r.StudentNo) AS TotalStudents FROM Registration r JOIN
Section s ON r.SectionID = s.SectionNo WHERE s.CourseId = 'CT220' AND r.Semester = 'I-
2012';

Q7 Solution

- i. The table is already in 1NF
- ii. The table is already in 2NF: No partial dependencies, entire pk determines each non-key attributes

FD exist:

ApptNo $\longrightarrow$ ApptDate, Appt Time, Appt Type, Patient ID, Fname, Lname, Doctor ID, Contact#, Dname

- iii. It has transitive dependencies.

FD's exist:

ApptNo $\longrightarrow$ ApptDate, Appt Time, Appt Type, Patient ID, Doctor ID

PatientID $\longrightarrow$ Fname, Lname, Contact#

DoctorID $\longrightarrow$ Dname

iv.

<u>Appt No</u>	Appt Date	Appt Time	Appt Type	Patient ID	Doctor ID
----------------	-----------	-----------	-----------	------------	-----------

<u>Patient ID</u>	Fname	Lname	Contact#
-------------------	-------	-------	----------

<u>Doctor ID</u>	Dname
------------------	-------

Q8 Solution

ENTITY: Customer

- Attributes: **PK**: CustomerID, Name, Email, PhoneNumber, ShippingAddress
- Relationships:
 - Places (1:N) → Order
 - Has(1:N) → ShippingAddress

ENTITY: Order

- Attributes: **PK**: OrderID, OrderDate, TotalPrice, OrderStatus, ShippingAddress, **FK**: CustomerID
- Relationships:
 - Contains (1:N) → OrderItem
 - Pays For (1:1) → Payment
 - Ships (1:1) → Shipping

ENTITY: OrderItem

- Attributes: **PK**: OrderItemID, Quantity, PriceAtPurchase, **FK**: OrderID, ProductID
- Relationships:
 - Refers To (N:1) → Product

ENTITY: Product

- Attributes: Name, Description, Price, Category, AvailableQuantity, **PK**: ProductID , **FK**: VendorID
- Relationships:
 - Sells (N:1) → Vendor
 - Tracks Availability Of (1:1) → Inventory

ENTITY: Vendor

- Attributes: Name, ContactInfo, ShippingPolicy, **PK**: VendorID
- Relationships:
 - Sells (1:N) → Product

ENTITY: Payment

- Attributes: PaymentMethod, PaymentAmount, PaymentStatus ,**PK**: PaymentID, **FK**: OrderID
- Relationships:
 - Pays For (1:1) → Order

ENTITY: Shipping

- Attributes: Carrier, ShippingCost, TrackingNumber, ShippingStatus, **PK**: ShippingID, **FK**: OrderID
- Relationships:
 - Ships (1:1) → Order
 - Ships_to (N:1) → ShippingAddress

ENTITY: Inventory

- Attributes: AvailableQuantity , **PK**: InventoryID, **FK**: ProductID
- Relationships:

Tracks Availability Of (1:1) → Product

ENTITY: ShippingAddress

- Attributes ShippingAddressID (**PK**), StreetAddress, City, State, PostalCode, Country, CustomerID (**FK**) – Linked to Customer

- Relationships:

Ships_to (1:N) → Shipping

